Oxford-IIIT Pet Semantic Segmentation Using a Pretrained U-Net

Rachael Chen

May 31, 2026

Abstract

This project explores semantic segmentation on the Oxford-IIIT Pet dataset using a U-Net based architecture. The task was to classify each pixel into one of three classes: foreground, background, and boundary. A pretrained ResNet34 encoder combined with a U-Net decoder was used to improve segmentation performance. Several improvements were explored, including weighted cross-entropy loss, proper multiclass mIoU evaluation, and best-model checkpoint saving. The final model achieved a public leaderboard score of 0.82131 on the Kaggle competition.

1 Introduction

Semantic segmentation is a computer vision task in which every pixel in an image is assigned a class label. In this project, the goal was to segment pet images from the Oxford-IIIT Pet dataset into foreground, background, and boundary regions.

The competition restricted participants to U-Net style architectures and allowed ImageNet pretrained encoders. Because segmentation requires both high-level semantic understanding and precise spatial localization, U-Net architectures are well suited for this task due to their encoder-decoder structure and skip connections.

2 Implementation

2.1 Dataset

The dataset used was the Oxford-IIIT Pet dataset provided through the competition. Each image contained a corresponding trimap mask. The original labels from the dataset used the convention:

- 1 = foreground
- 2 = background
- 3 = boundary

These labels were remapped to:

- 0 = foreground
- 1 = background
- 2 = boundary

Images were resized to 224×224 and normalized using ImageNet normalization statistics. The dataset was split into:

- Training set: 6614 images
- Validation set: 735 images

2.2 Model Architecture

The final model used a pretrained ResNet34 encoder with a U-Net decoder implemented using the segmentation$_{m}odels_{p}ytorchlibrary$.

The encoder extracts increasingly abstract visual features through convolution and pooling operations. Early layers learn edges and textures, while deeper layers learn semantic information such as animal shapes and body regions.

The decoder upsamples these compressed feature maps back to the original image resolution to generate dense pixel-wise predictions. Skip connections between encoder and decoder layers help preserve spatial information and improve segmentation boundary quality.

The final layer outputs three channels corresponding to the three segmentation classes.

2.3 Training Procedure

The model was trained using the Adam optimizer with a learning rate of 1×10^{-4} for 50 epochs.

A weighted cross-entropy loss was used to address class imbalance, especially because boundary pixels occupy a relatively small portion of the image. The boundary class was assigned a higher weight:
[[1.0, 1.0, 3.0]]

Validation performance was measured using mean Intersection-over-Union (mIoU) across all three classes.

The best model checkpoint was saved during training based on validation mIoU.

2.4 Experiments and Improvements

The initial implementation used a custom U-Net architecture trained from scratch. While functional, this model achieved limited performance.

Several improvements significantly increased segmentation quality:

- Replacing the custom encoder with a pretrained ResNet34 encoder
- Correcting the mIoU calculation to include all three classes
- Using weighted cross-entropy loss
- Saving the best checkpoint instead of the final epoch
- Increasing the number of training epochs

The pretrained encoder provided the largest improvement because the model could leverage visual features learned from ImageNet instead of learning all low-level image representations from scratch.

2.5 Results

The final model achieved a public leaderboard score of 0.82131.

Qualitatively, the pretrained U-Net produced significantly cleaner masks and more accurate boundaries compared to the original implementation. The model generalized reasonably well even on out-of-distribution animal categories included in the test set.

2.6 Discussion

This project demonstrated the effectiveness of transfer learning for semantic segmentation tasks. The pretrained encoder substantially improved both convergence speed and final performance.

One limitation of the project was the relatively limited experimentation with augmentations and alternative losses such as Dice loss. Additional improvements could likely be achieved using U-Net++, stronger augmentations, or larger encoder backbones such as ResNet50.

Overall, the project showed how architectural choices, evaluation metrics, and training procedures strongly influence segmentation performance.

2.7 References

- Oxford-IIIT Pet Dataset
- segmentation$_{m}odels_{p}ytorchlibrary$
- PyTorch documentation
- ChatGPT was used for debugging assistance, architecture explanations, and implementation guidance.